

SOC LAB WRITE-UP

Building a Mini SIEM and Security Operations Center Lab

Generating Synthetic Security Logs, Engineering Detections, Evaluating Alerts, and Investigating Threat Activity

Project Category	Defensive Security and Detection Engineering
Primary Objective	Build a reproducible SOC workflow that collects, normalizes, detects, evaluates, and presents security events
Data Sources	Windows security events, Linux authentication logs, and firewall records
Detection Coverage	Eight attack classes mapped to MITRE ATT&CK techniques
Test Window	September 22 through September 24, 2026

Prepared by Caleb James

Cybersecurity Project Portfolio

Table of Contents

1. Executive Summary
2. Problem Statement and Objectives
3. SOC Architecture and Environment
4. Tools and Technologies
5. Methodology
 - 5.1 Synthetic Log Generation
 - 5.2 Log Collection and Normalization
 - 5.3 Detection Engineering
 - 5.4 Alert Evaluation
 - 5.5 Analyst Dashboard
6. Detection Coverage
7. Results
8. Risk Analysis and Detection Tuning
9. Lessons Learned
10. Deliverables and Impact
11. Conclusion
- Appendix A - Detection Rule Reference
- Appendix B - References

1. Executive Summary

This project was designed to reproduce the core workflow of a small security operations center in a controlled environment. The objective was to move beyond simply writing isolated detection rules and instead build a complete pipeline that produces security telemetry, centralizes that telemetry, identifies suspicious behavior, measures detection quality, and presents the resulting alerts in a form an analyst could investigate. This approach follows the broader purpose of log management described by NIST, where collected event data supports incident identification, investigation, operational troubleshooting, and retention requirements [1].

The environment uses synthetic activity from two Windows workstations, two Linux servers, and a firewall. The generator creates normal background activity and sixteen labeled attack campaigns across eight attack classes. The labels are separated from the operational fields consumed by the detection engine, which prevents the detector from simply reading the answer key. The resulting data represents approximately 1,150 benign events plus the injected attack activity.

A collection stage parses three different raw formats and places them into a common SQLite events table. This gives the detection engine a consistent schema for timestamps, hosts, users, source addresses, destination addresses, ports, event types, and other evidence. Centralizing logs is also consistent with guidance from CISA and the Center for Internet Security, both of which emphasize collecting and reviewing logs from multiple assets in a centralized process [2] [3].

The detection engine contains eight independent detectors covering brute force authentication, suspicious PowerShell, port scanning, impossible login behavior, privilege escalation, new administrator accounts, persistence mechanisms, and large outbound transfers. Each alert contains severity, evidence, a MITRE ATT&CK mapping, a confidence value, and a recommended response. The project therefore connects raw telemetry to analyst-oriented findings rather than stopping at simple event matching.

The final evaluation detected all sixteen injected campaigns. The engine generated twenty-nine alerts, of which twenty-eight matched a real campaign. This produced campaign-level recall of 100 percent, alert-level precision of approximately 0.97, and an F1 score of approximately 0.98. The remaining false positive was useful because it exposed an alert-correlation issue where two detectors responded to the same underlying activity. That result demonstrates why detection quality cannot be judged by detection count alone.

2. Problem Statement and Objectives

A security operations team must convert a large volume of routine system activity into a smaller set of actionable findings. The challenge is not simply collecting logs. Different operating systems and network devices produce different formats, useful evidence may be spread across several events, and individual events often become meaningful only when examined within a time window or in relation to other activity. NIST guidance emphasizes that effective incident response depends on analyzing incident-related information and integrating response activities into broader cybersecurity risk management [4].

The project addressed this problem by building a compact detection pipeline that could be tested repeatedly under controlled conditions. Instead of relying on real organizational data, the lab creates

synthetic telemetry with known attack campaigns. This provides a measurable ground truth while keeping the exercise isolated and safe.

2.1 Objectives

- Generate realistic background activity across Windows, Linux, and firewall sources.
- Inject repeatable attack campaigns representing common SOC detection scenarios.
- Normalize different log formats into a common event model.
- Develop independent rule-based detectors using temporal, behavioral, and content-based conditions.
- Map detections to MITRE ATT&CK techniques and tactics.
- Evaluate detections against an independent ground-truth answer key.
- Measure recall, precision, false positives, and F1 rather than relying only on whether an alert appeared.
- Present alert evidence and response guidance through a self-contained analyst dashboard.

2.2 Success Criteria

The project was considered successful if the pipeline could process all generated sources without manual restructuring, identify the injected attack campaigns, provide enough evidence for an analyst to understand why an alert fired, and expose measurable weaknesses through evaluation. A strong result therefore required both broad detection coverage and transparent reporting of false positives.

3. SOC Architecture and Environment

The lab models a small enterprise environment with multiple telemetry producers feeding a central analysis workflow. The design deliberately separates data generation, collection, detection, evaluation, and presentation. This separation makes it possible to change one stage without redesigning the others.

Source	Representative Role	Raw Format	Security Value
Windows PC 01 and 02	Endpoint activity	Pipe-delimited security events	Authentication, PowerShell, account and privilege events
Linux Server 01 and 02	Server activity	Syslog and auth-style records	SSH, sudo, account, cron, and key activity
Firewall	Network telemetry	CSV-style connection records	Connections, ports, allow or deny outcomes, transfer size
Ground truth	Evaluation reference	JSON	Campaign identity and timing for scoring only

The architecture intentionally resembles the centralized collection model used in production security monitoring. NIST describes log management as the generation, transmission, storage, access, and disposal of event records, while CISA recommends enabling logging on servers, endpoints, network devices, and cloud services and centralizing the results for analysis [1] [2].

3.1 Data Flow

Windows	Linux	Firewall	Collector	SQLite Events	Detection Engine	Evaluation and Dashboard
---------	-------	----------	-----------	---------------	------------------	--------------------------

The flow begins with heterogeneous event sources. The collector parses those records and writes normalized events into SQLite. The detection engine reads only operational fields and creates alerts. The evaluation stage compares alerts against the separate answer key. The dashboard then combines alert data and scorecard metrics into an analyst-oriented interface.

3.2 Data Model

Field Group	Examples	Purpose
Time	ts	Supports sliding windows and event correlation
Identity	user, source IP	Links activity to an account or origin
Asset	host, destination IP	Identifies the affected system
Network	destination port, bytes	Supports scanning and transfer detections
Event context	event type, detail, raw	Preserves evidence for analyst review
Evaluation metadata	gt_id	Used only by the scoring stage

The inclusion of a separate evaluation identifier is important. The collector preserves the ground-truth identifier for scoring, but the detectors do not use that field. This prevents the evaluation label from becoming a hidden detection feature and preserves the distinction between detection and measurement.

4. Tools and Technologies

Technology	Role in the Project	Reason for Use
Python	Pipeline implementation	Provides data generation, parsing, detection logic, evaluation, and dashboard generation
SQLite	Central event store	Provides a lightweight relational structure for normalized events and alerts
Regular expressions	Log parsing and content matching	Extracts structured values from heterogeneous text records
JSON	Ground truth and alert exchange	Provides portable structured data for evaluation and presentation
HTML and CSS	Analyst dashboard	Produces a self-contained interface without a server dependency
MITRE ATT&CK	Technique mapping	Provides a common vocabulary for describing adversary behavior

The choice of lightweight technologies keeps the lab reproducible while still demonstrating the main engineering concepts behind a SIEM workflow. The project does not attempt to replicate the scale, storage architecture, or enrichment capabilities of a commercial platform. Its purpose is to expose the reasoning that occurs between raw telemetry and an analyst decision.

5. Methodology

5.1 Synthetic Log Generation

The first stage creates three days of activity from September 22 through September 24, 2026. A fixed random seed makes the generation repeatable. The background workload includes Windows logons,

failed password typos, normal PowerShell commands, scheduled tasks, Linux SSH activity, routine sudo commands, cron jobs, ordinary firewall connections, and moderate outbound transfers.

The attack layer injects sixteen campaigns across eight categories. The campaigns include brute force authentication, suspicious PowerShell, port scanning, impossible login behavior, privilege escalation, new administrator accounts, persistence mechanisms, and large outbound transfers. Each campaign records an attack type, start and end time, source, target, MITRE technique, and explanatory note in the ground-truth file.

The design intentionally separates the labels from the information used by the detector. Attack records contain a ground-truth identifier during generation, but the collector removes that marker from the operational event content while retaining it in a separate field. This allows the evaluation stage to know what happened without allowing the detection engine to know the answer.

The attack scenarios also use realistic event relationships. For example, the brute force scenario generates repeated failed authentication followed by a successful authentication, while the new administrator scenario creates an account and then adds it to an administrative group. Microsoft documents Windows event 4624 as a successful logon event and identifies related security events as useful monitoring sources, supporting the project choice to build authentication detections around these event patterns [5].

5.2 Log Collection and Normalization

The collector reads the Windows, Linux, and firewall records independently and converts them into a common events table. Windows records are split into event code, event type, user, and source address fields. Linux records are parsed with patterns for SSH authentication, sudo commands, account creation, group changes, cron activity, and authorized key changes. Firewall records are converted into source address, destination address, destination port, action, and byte count fields.

This normalization stage is central to the project because the detection engine should not need separate logic for every source format. Instead, detectors operate on shared concepts such as a failed logon, a PowerShell event, a destination port, or a large transfer. This reflects the purpose of centralized log management described by NIST and the CIS Controls, which call for central collection so events from different assets can be analyzed together [1] [3].

5.3 Detection Engineering

The detection engine contains eight independent rules. Each detector examines a specific behavioral pattern and creates an alert when its conditions are satisfied. The rules use sliding windows, event sequencing, content matching, baseline comparison, and contextual factors such as business hours.

The brute force detector requires at least eight failed logons from one source against one host within five minutes. A successful logon shortly afterward raises the severity to Critical. MITRE ATT&CK identifies brute force as technique T1110 and describes repeated credential guessing as a method adversaries can use to obtain account access [6].

The suspicious PowerShell detector looks for patterns associated with encoded commands, hidden execution, remote download cradles, Defender exclusion changes, credential dumping references, privilege enumeration, and execution policy bypass. The project maps this activity to PowerShell under ATT&CK technique T1059.001.

The port scanning detector looks for at least fifteen distinct destination ports against a single target within two minutes. Severity increases when most of the observed connections are denied. The impossible login detector identifies successful authentication from different geographic regions within sixty minutes. The project maps that behavior to Valid Accounts because compromised legitimate credentials can be abused to obtain or maintain access [7].

The privilege escalation detector uses different logic for Linux and Windows. Linux activity is correlated around sudo access and subsequent sudoers or SUID modification. Windows activity looks for privilege enumeration followed by a special privilege assignment. New administrator account detection links account creation to immediate administrative group membership. Persistence detection looks for scheduled tasks, services, cron jobs, and SSH key changes associated with suspicious writable paths.

The large outbound transfer detector compares a transfer to the network's ninety-fifth percentile baseline and requires a minimum size floor. This approach is intended to reduce alerts caused by normal moderate transfers while highlighting unusually large movement of data, especially outside normal business hours.

5.4 Alert Evaluation

Evaluation is treated as a separate measurement problem rather than part of the detection logic. The evaluator loads the ground-truth campaigns and generated alerts, then matches them using attack type, temporal overlap, and a shared source address or target host. A two-minute grace window is used to account for the difference between campaign timing and the moment when a rule fires.

The evaluator also handles a correlation edge case in which multiple campaigns of the same type occur close together. Instead of assigning an alert to the first matching campaign, it assigns the alert to the candidate with the closest campaign start time. This prevents a simplistic first-match strategy from creating an artificial miss.

The project reports campaign-level recall, alert-level precision, false positives, false negatives, and F1. Using multiple measurements is important because a detection system can produce many alerts while still performing poorly. NIST's incident response guidance emphasizes analyzing incident information and improving the effectiveness of detection and response activities, which aligns with the project's decision to measure both coverage and alert quality [4].

5.5 Analyst Dashboard

The final stage builds a self-contained HTML dashboard from the alert and evaluation data. The interface provides summary metrics, a scorecard, alert filtering, severity information, evidence, MITRE mappings, and recommended response actions. The dashboard is designed to resemble a compact analyst console rather than a raw program output.

The dashboard separates the evidence from the measurement layer. Analysts can review individual alerts and the overall scorecard without interacting with the underlying Python implementation. This supports the practical goal of turning detection-engineering results into information that could be consumed during triage.

6. Detection Coverage

Detection	Behavior	Window or Condition	ATT&CK	Severity Logic
Brute force authentication	Repeated failed logons	8 or more failures within 5 minutes	T1110	Critical after successful login, otherwise High
Suspicious PowerShell	Encoded or evasive PowerShell	One or more suspicious patterns	T1059.001	High or Critical based on pattern count
Port scanning	Many destination ports	15 or more ports within 2 minutes	T1046	High when deny ratio exceeds 50 percent
Impossible login	Rapid geographic change	Different regions within 60 minutes	T1078	Critical
Privilege escalation	Sudo or token privilege change	Related events within 5 minutes	T1548.003 and T1134	High or Critical
New administrator account	Account creation followed by admin membership	Within 1 to 2 minutes	T1136.001	Critical after hours, otherwise High
Persistence mechanisms	Suspicious task, service, cron, or key change	Suspicious writable path or key change	T1053.x, T1543.003, T1098.004	High
Large outbound transfer	Potential exfiltration	3 times 95th percentile and at least 400 MB	T1048	Higher outside business hours

The coverage demonstrates several common detection-engineering patterns. Some rules are event-content rules, some are threshold rules, and others depend on sequences of related events. This variety is important because a SOC cannot rely on one detection style for every threat.

The ATT&CK mappings also provide context for why the behaviors matter. MITRE describes network and credential behaviors as adversary techniques that can appear across multiple stages of an intrusion. Mapping the rules to that vocabulary makes the alerts easier to interpret and provides a foundation for future detection coverage [6] [7].

7. Results

The test run covered sixteen injected attack campaigns against approximately 1,150 benign background events. Every injected campaign produced at least one alert. The engine generated twenty-nine alerts in total, with twenty-eight matching a real campaign.

Metric	Result	Interpretation
Attack campaigns	16	Known malicious campaigns injected into the synthetic environment
Benign background events	Approximately 1,150	Normal activity used to provide detection context
Total alerts	29	All alerts generated by the eight detectors
Detected campaigns	16 of 16	Campaign-level recall of 100 percent
True-positive alerts	28 of 29	Alerts matching a real injected campaign
Alert-level precision	0.97	Approximately 97 percent of alerts mapped to a campaign
F1 score	0.98	Combined precision and recall measure
False positives	1	An overlap between privilege escalation and PowerShell logic

7.1 False Positive Investigation

The single false positive was not caused by random noise. A Windows privilege escalation campaign included the command `whoami /priv`. The suspicious PowerShell detector treats privilege enumeration as one suspicious pattern, while the privilege escalation detector separately uses the same event as

part of a sequence that ends in a special privilege assignment. Both detectors therefore produced alerts from the same underlying activity.

This finding is valuable because it identifies a real tuning problem. A production SIEM would normally correlate related alerts into one incident so that analysts are not asked to investigate the same activity twice. The lab intentionally leaves the overlap visible so that alert correlation remains measurable rather than being hidden inside the evaluator.

7.2 Evaluation Edge Case

Two outbound transfer campaigns occurred within two minutes of each other on the same host. A simple first-match scoring method could assign an alert to the wrong campaign and manufacture a false negative. The evaluator instead selects the temporally closest matching campaign. This demonstrates that evaluation logic can influence reported detection performance and therefore deserves the same care as detection logic.

7.3 What the Results Demonstrate

- The pipeline can detect every injected campaign in the controlled scenario.
- The engine produces relatively few false positives under the synthetic workload.
- Detection rules benefit from temporal correlation rather than isolated event matching.
- Detection performance depends on both rule quality and evaluation methodology.
- The remaining false positive provides a concrete target for future alert correlation and tuning.

8. Risk Analysis and Detection Tuning

The results should be interpreted as evidence about the synthetic scenario rather than proof of production readiness. The environment contains deliberately structured attacks, a limited number of hosts, known log formats, and controlled timing. Real enterprise telemetry contains much greater variability, incomplete data, duplicated events, clock drift, administrative exceptions, changing baselines, and adversary behavior that may not resemble the injected patterns.

Observed Issue	Security or Operational Risk	Recommended Improvement
Overlapping detectors	One underlying event can create multiple alerts	Add alert deduplication and incident correlation
Static thresholds	Normal activity can change over time	Tune thresholds using asset and user baselines
Synthetic geography	Real identity systems have richer location context	Use trusted identity and network context
Limited telemetry	Important evidence may be absent	Add process creation, DNS, endpoint, identity, and cloud logs
Fixed evaluation window	Timing differences can affect attribution	Use campaign-aware correlation with clock tolerance
Rule-based content matching	Obfuscation can bypass simple patterns	Add behavioral signals and layered detections

The tuning priorities also align with established security guidance. CISA recommends monitoring logs regularly, establishing alerts for high-risk events, protecting logs from unauthorized modification, and retaining records according to organizational requirements [2]. CIS Control 8 similarly emphasizes collecting, alerting, reviewing, and retaining audit logs that help organizations detect, understand, and recover from attacks [3].

The project also fits naturally into the NIST Cybersecurity Framework 2.0 model. Logging and detection contribute to the Detect function, while the alert evidence and recommended response fields support the transition into Respond activities. NIST describes CSF 2.0 as a framework for helping organizations understand and improve management of cybersecurity risk [8].

9. Lessons Learned

- A useful SIEM exercise must model the entire path from telemetry to analyst action rather than focusing only on alert generation.
- Normalization is one of the most important stages because detection logic becomes simpler when different sources expose a common event model.
- Temporal correlation provides stronger evidence than many single-event rules. Several detections in this project depend on what happens before or after an event.
- Ground truth must remain separate from detector input. Otherwise, a test can appear highly accurate while quietly using information that would not exist in a real SOC.
- False positives are valuable engineering feedback. The single overlap in this project identified a concrete opportunity for alert deduplication and incident correlation.
- Evaluation logic deserves testing of its own. The close exfiltration campaigns showed that a weak scoring method can create artificial misses even when the detector behaves correctly.
- MITRE ATT&CK mapping improves analyst context because a raw rule name becomes connected to a recognized adversary behavior.
- A dashboard should explain why an alert fired and what an analyst could do next. A severity label without evidence or response guidance provides limited operational value.
- Synthetic environments are useful for controlled measurement, but their results should not be treated as a substitute for testing against diverse real telemetry.

10. Deliverables and Impact

The project produced more than a collection of scripts. The completed workflow provides a repeatable security monitoring exercise with data generation, centralized event storage, eight detection rules, an evaluation system, structured alert output, and an analyst dashboard.

Deliverable	Outcome
Synthetic telemetry generator	Creates repeatable benign activity and sixteen labeled attack campaigns
Log collector	Converts Windows, Linux, and firewall formats into a common event schema
SQLite event store	Provides centralized access to normalized events and generated alerts
Detection engine	Implements eight independent behavioral detectors with ATT&CK mappings
Evaluation engine	Measures recall, precision, false positives, false negatives, and F1
Analyst dashboard	Presents alerts, scorecards, evidence, severity, and response guidance
Detection reference	Documents the logic and ATT&CK coverage for each detector

The practical impact of the project is the experience gained in detection engineering and SOC analysis. The exercise required thinking about what evidence exists in logs, how events relate to one another, how a rule can generate noise, and how to determine whether an alert actually corresponds to malicious behavior. These are more transferable skills than memorizing individual SIEM commands.

The project also creates a foundation for future expansion. Additional telemetry could be added without redesigning the entire workflow if it can be normalized into the shared event model. New detectors can consume the same normalized events, while the evaluation layer can add new attack campaigns and scoring criteria.

11. Conclusion

This project achieved its core objective of building a compact but complete SOC detection workflow. It began with synthetic multi-source activity, centralized that activity into a normalized event store, applied eight detection strategies, evaluated the results against an independent answer key, and presented the findings through an analyst dashboard.

The final test detected all sixteen injected campaigns and generated twenty-nine alerts, with twenty-eight corresponding to real campaigns. The resulting campaign-level recall of 100 percent, alert-level precision of approximately 0.97, and F1 score of approximately 0.98 show that the detection rules performed well against the defined scenarios. At the same time, the remaining false positive and the close-timing evaluation case demonstrated why correlation and measurement deserve continued attention.

Most importantly, the project converted security monitoring concepts into an end-to-end engineering exercise. It demonstrates how raw logs become normalized security events, how events become detection signals, how signals become analyst alerts, and how those alerts can be measured and improved. That workflow provides a practical foundation for further work in SOC operations, detection engineering, threat hunting, and incident response.

Appendix A - Detection Rule Reference

ATT&CK Mapping	Detector	Rule Summary
T1110	Brute force authentication	Eight or more failed logons from one source against one host within five minutes. A following success raises severity.
T1059.001	Suspicious PowerShell	Flags encoded execution, hidden execution, remote download behavior, Defender exclusion changes, credential tool references, privilege enumeration, and bypass patterns.
T1046	Port scanning	Detects fifteen or more distinct destination ports against one target within two minutes and considers the deny ratio.
T1078	Impossible login	Detects successful authentication for one account from different geographic regions within sixty minutes.
T1548.003 and T1134	Privilege escalation	Correlates Linux sudo and privilege modification activity or Windows privilege enumeration with a special privilege assignment.
T1136.001	New administrator accounts	Correlates account creation with immediate administrative group membership and increases severity outside business hours.
T1053.x, T1543.003, T1098.004	Persistence mechanisms	Looks for suspicious scheduled tasks, services, cron jobs, and SSH key modifications associated with writable or hidden paths.
T1048	Large outbound transfers	Flags transfers at least three times the network ninety-fifth percentile baseline with a minimum threshold of 400 MB

Appendix B - References

- [1] National Institute of Standards and Technology. Guide to Computer Security Log Management, Special Publication 800-92, 2006.
- [2] Cybersecurity and Infrastructure Security Agency. Use Logging on Business Systems, current guidance accessed September 2026.
- [3] Center for Internet Security. CIS Controls v8.1, Control 8 Audit Log Management, 2025.
- [4] National Institute of Standards and Technology. Incident Response Recommendations and Considerations for Cybersecurity Risk Management, Special Publication 800-61 Revision 3, 2025.
- [5] Microsoft. Windows Security Auditing Event 4624 Account Successfully Logged On, Microsoft Learn, accessed September 2026.
- [6] MITRE. ATT&CK Technique T1110 Brute Force, Enterprise ATT&CK, updated May 2026.
- [7] MITRE. ATT&CK Technique T1078 Valid Accounts, Enterprise ATT&CK, accessed September 2026.
- [8] National Institute of Standards and Technology. Cybersecurity Framework 2.0, 2024.